

# SISTEMA ACADEMICO

Gestion de estudiantes, materias y notas - Documentacion completa del código fuente

## 1. QUE HACE ESTE SISTEMA

Aplicacion de escritorio escrita en Python para llevar el control de alumnos, materias y calificaciones de una institucion educativa. Tiene dos tipos de usuario: el docente, que registra y modifica todo, y el estudiante, que unicamente consulta sus propias notas. No usa base de datos: toda la informacion se guarda en archivos JSON dentro de la carpeta datos.

## 2. REQUISITOS

- Python 3.10 o superior (probado en 3.13).
- Tkinter: viene incluido con Python en Windows y macOS.
- En Linux puede hacer falta instalarlo aparte: `sudo apt install python3-tk`
- Libreria reportlab: solo para generar este PDF (`pip install reportlab`).
- La aplicacion NO usa base de datos. Los datos se guardan en archivos JSON.
- La aplicacion NO necesita instalar nada mas. No hay `pip install` para ella.

## 3. COMO EJECUTARLO

1) Abre la carpeta `sistema_academico`.  
2) Abre una terminal (CMD o PowerShell) en esa carpeta.  
3) Escribe: `python principal.py`  
4) Inicia sesion con `admin` / `admin123`.

*En este documento el código aparece en el ORDEN EXACTO en el que se debe crear cada archivo. Empieza por `config.py` y termina en `principal.py`.*

## 4. GUIA PASO A PASO PARA CONSTRUIR EL PROYECTO

*Sigue estos pasos en orden. Cada archivo depende del anterior, por eso importarlos en desorden produce error de modulo no encontrado.*

1. Crea la carpeta `sistema_academico` y dentro las carpetas `nucleo`, `interfaz`, `datos` y `documentacion`.
2. Crea los archivos vacios `nucleo/__init__.py` e `interfaz/__init__.py`. Sirven para que Python reconozca las carpetas como paquetes y los imports funcionen.
3. Crea `config.py`. Aqui se decide en que carpeta se guardan los JSON.
4. Crea `nucleo/seguridad.py`. Debe estar antes que los demas porque `servicios.py` lo usa para las contraseñas.
5. Crea `nucleo/almacen.py`. La clase `AlmacenJSON` es la que sabe leer y escribir un archivo de datos.
6. Crea `nucleo/servicios.py`. Es la capa con todas las reglas: aqui se validan los datos y se comprueba el rol de quien esta usando el programa.
7. Crea `interfaz/estilos.py`. Define el tema para que todas las pantallas se vean iguales.
8. Crea `interfaz/widgets.py`. Los componentes que se reutilizan en todos los formularios.
9. Crea `interfaz/vista_login.py`. La primera pantalla que ve el usuario.
10. Crea `interfaz/dialogos.py`. Los formularios de estudiante, materia, nota y contraseña.
11. Crea `interfaz/vista_docente.py`. El panel con las tres pestanas.
12. Crea `interfaz/vista_estudiante.py`. El panel de solo lectura.
13. Crea `principal.py` al final, porque es el que importa todos los modulos anteriores.
14. Abre una terminal dentro de la carpeta `sistema_academico` y ejecuta: `python principal.py`
15. Inicia sesion con el usuario `admin` y la contraseña `admin123`. Esa cuenta de docente se crea sola la primera vez.

## 5. ESTRUCTURA DEL PROYECTO

Este es el arbol de carpetas y archivos del proyecto terminado:

```
sistema_academico/
|-- principal.py ..... Ventana principal y control de vistas
|-- config.py ..... Rutas de los JSON y constantes
|-- README.md ..... Texto de ayuda del proyecto
|-- nucleo/
| |-- __init__.py ..... Paquete vacio
| |-- seguridad.py ..... Hash de contraseñas (PBKDF2 + sal)
| |-- almacen.py ..... Lectura y escritura de JSON
| `-- servicios.py ..... Reglas de negocio, validaciones, permisos
|-- interfaz/
| |-- __init__.py ..... Paquete vacio
| |-- estilos.py ..... Colores y estilos de ttk
| |-- widgets.py ..... Campos de formulario, tablas, tarjetas
| |-- vista_login.py ..... Pantalla de inicio de sesion
| |-- dialogos.py ..... Formularios de estudiante, materia, nota
| |-- vista_docente.py ..... Panel completo del docente
| `-- vista_estudiante.py ..... Panel de consulta del estudiante
|-- datos/
| |-- usuarios.json ..... Cuentas de acceso (docentes y alumnos)
| |-- estudiantes.json ..... Fichas de los estudiantes
| |-- materias.json ..... Materias del plan de estudios
| `-- notas.json ..... Calificaciones
`-- documentacion/
   |-- generar_pdf.py ..... Este generador de documentacion
```

### Que hace cada archivo

| Archivo                      | Funcion dentro del sistema                                                                                |
|------------------------------|-----------------------------------------------------------------------------------------------------------|
| principal.py                 | Arranca el programa. Crea la ventana, aplica el tema y muestra la vista de login o el panel segun el rol. |
| config.py                    | Define donde viven los archivos JSON y los limites validos de notas, creditos y semestres.                |
| nucleo/seguridad.py          | Protege las contraseñas con sal y hash PBKDF2. Ninguna contraseña se guarda en texto plano.               |
| nucleo/almacen.py            | Guarda y lee los datos en JSON. Si un archivo se dania, avisa en lugar de perder la informacion.          |
| nucleo/servicios.py          | Valida los datos y aplica las reglas. Aqui se bloquea que un estudiante modifique calificaciones.         |
| interfaz/estilos.py          | El tema: colores, fuentes y estilos de los controles de Windows.                                          |
| interfaz/widgets.py          | Los piezas reutilizables de los formularios.                                                              |
| interfaz/vista_login.py      | La pantalla donde se escribe usuario y contraseña.                                                        |
| interfaz/dialogos.py         | Las ventanas de formulario de estudiante, materia, nota y cambio de contraseña.                           |
| interfaz/vista_docente.py    | El tablero de trabajo del docente.                                                                        |
| interfaz/vista_estudiante.py | La pantalla de consulta del alumno.                                                                       |
| datos/*.json                 | La informacion del sistema. Se puede copiar como respaldo o abrir con cualquier editor de texto.          |

## 6. PASO A PASO DE USO

### 6.1 El docente

- Entra con **admin / admin123** (o la cuenta de docente que te den).
- En la pestana **Materias** pulsa **+ Nueva materia** y registra las materias: codigo, nombre, creditos y semestre.
- En la pestana **Estudiantes** pulsa **+ Nuevo estudiante**. Llena cedula, nombres, apellidos, email, telefono y carrera.
- Deja vacios **Usuario de acceso** y **Contraseña inicial** y el programa los genera. Anota los datos: se muestran una sola vez.
- En la pestana **Notas** pulsa **+ Nueva nota**, elige estudiante y materia, y escribe la calificación de 0 a 100.
- Usa el campo **Buscar** de cada pestana para filtrar por nombre, código, carrera o periodo.
- Selecciona una fila y pulsa **Editar** o **Eliminar**. Si borras un estudiante o una materia, sus notas se borran también.
- Si un alumno pierde su clave, seleccionalo y pulsa **Restablecer clave**.

### 6.2 El estudiante

- Entra con el usuario y la contraseña que te dio el docente.
- Pestana **Todas mis notas**: lista completa con el estado Aprobado o Reprobado en cada fila. Se considera aprobado desde 60 puntos.
- Pestana **Por materia**: el promedio de cada materia y sus créditos.
- Pestana **Mi perfil**: cedula, carrera, email y telefono.
- El alumno no puede agregar, editar ni borrar nada. Solo consultar.

## 7. ROLES Y PERMISOS

*Esta regla no depende de que el boton este oculto: la comprobacion esta en nucleo/servicios.py, asi que ningun estudiante puede modificar calificaciones aunque se llame al metodo directamente.*

| Accion                         | Docente | Estudiante |
|--------------------------------|---------|------------|
| Ver todos los estudiantes      | Si      | No         |
| Ver sus propias notas          | Si      | Si         |
| Agregar estudiantes            | Si      | No         |
| Editar estudiantes             | Si      | No         |
| Eliminar estudiantes           | Si      | No         |
| Agregar materias               | Si      | No         |
| Editar materias                | Si      | No         |
| Eliminar materias              | Si      | No         |
| Agregar notas                  | Si      | No         |
| Editar notas                   | Si      | No         |
| Eliminar notas                 | Si      | No         |
| Cambiar su propia contrasena   | Si      | Si         |
| Restablecer clave de un alumno | Si      | No         |

## 8. ESTRUCTURA DE LOS ARCHIVOS JSON

Cada archivo JSON es una lista de objetos. El campo `id` sirve para enlazar los datos: la nota guarda el `id` del estudiante y el `id` de la materia. Para respaldar el sistema solo hay que copiar la carpeta `datos`.

### usuarios.json

```
[
  {
    "id": "USR-0001",
    "nombre_usuario": "admin",
    "sal": "a3f9c1b2d4e5f60718293a4b5c6d7e8f",
    "clave_hash": "8c1d... (hash de la contrasena, no la contrasena)",
    "rol": "docente",
    "nombre": "Docente Principal",
    "estudiante_id": null
  },
  {
    "id": "USR-0002",
    "nombre_usuario": "anago",
    "sal": "9b2e4d7f1a3c5e8b0d2f4a6c8e0b1d3f",
    "clave_hash": "4e7a... (hash de la contrasena)",
    "rol": "estudiante",
    "nombre": "Ana Maria Gomez",
    "estudiante_id": "EST-0001"
  }
]
```

### estudiantes.json

```
[
  {
    "id": "EST-0001",
    "cedula": "1-2345678-9",
    "nombres": "Ana Maria",
    "apellidos": "Gomez Ruiz",
    "email": "ana@correo.com",
    "telefono": "0991234567",
    "carrera": "Ingenieria de Sistemas",
    "activo": true,
    "fecha_registro": "2026-10-02"
  }
]
```

### materias.json

```
[
  {
    "id": "MAT-0001",
    "codigo": "MAT101",
    "nombre": "Algebra Lineal",
    "creditos": 4,
    "semestre": 1,
    "docente": "Docente Principal"
  }
]
```

### notas.json

```
[
{
  "id": "NOT-0001",
  "estudiante_id": "EST-0001",
  "materia_id": "MAT-0001",
  "valor": 85.0,
  "tipo": "Parcial 1",
  "periodo": "2026-1",
  "fecha": "2026-10-02"
}
```

## 9. ERRORES CORREGIDOS DURANTE EL DESARROLLO

*Se documentan porque explican por que el código esta escrito de cierta manera y son los errores mas comunes al hacer un formulario con Tkinter.*

1. La tarjeta del login se dibujaba de 1 pixel de alto porque se combinaba `place(relwidth)` con `pack_propagate(False)`. Se uso un alto calculado segun el contenido.
2. El formulario de estudiante perdía 199 pixels de la parte de abajo y los botones Guardar y Cancelar ni siquiera se mostraban. La causa era que la geometria se fijaba antes de agregar los campos, y el cuerpo del dialogo absorbía todo el espacio dejando el pie en cero.
3. Al editar una nota se perdían las referencias del estudiante y de la materia, y la nota quedaba huérfana.
4. Las 15 llamadas a `messagebox` tenían el título y el mensaje invertidos, porque Tkinter recibe el título como palabra clave y no como posición.
5. El filtro de notas no buscaba por el código de la materia, solo por el nombre.
6. Si un registro del JSON venía sin el campo docente, el programa se caía con un error de clave. Ahora los registros incompletos se saltan.

## 10. CODIGO COMPLETO DE CADA ARCHIVO

*Cada archivo se explica y luego se muestra su contenido integro. Copia el bloque dentro de un archivo con el mismo nombre y la misma carpeta.*

### 10.1 Configuracion global

Ruta: [sistema\\_academico/config.py](#)

*Rutas absolutas de los cuatro archivos JSON y constantes de validacion. Es el unico lugar donde hay que cambiar rutas si el proyecto se mueve.*

17 lineas

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
DIRECTORIO_DATOS = BASE_DIR / "datos"

ARCHIVO_USUARIOS = DIRECTORIO_DATOS / "usuarios.json"
ARCHIVO_ESTUDIANTES = DIRECTORIO_DATOS / "estudiantes.json"
ARCHIVO_MATERIAS = DIRECTORIO_DATOS / "materias.json"
ARCHIVO_NOTAS = DIRECTORIO_DATOS / "notas.json"

TITULO_APP = "Sistema Academico"
VERSION = "1.0.0"

NOTA_MINIMA = 0.0
NOTA_MAXIMA = 100.0
CREDITOS_MINIMOS = 1
CREDITOS_MAXIMOS = 10
```

### 10.2 Paquete nucleo (vacio)

Ruta: [sistema\\_academico/nucleo/\\_\\_init\\_\\_.py](#)

*Marca la carpeta como paquete de Python. No lleva codigo.*

*(archivo vacio de proposito)*

### 10.3 Seguridad de contraseñas

Ruta: [sistema\\_academico/nucleo/seguridad.py](#)

*Genera la sal, calcula el hash PBKDF2-SHA256 y compara contraseñas en tiempo constante. Las contraseñas nunca se guardan en texto plano.*

30 lineas

```

import hashlib
import secrets

ITERACIONES = 120_000
LONGITUD_SAL = 16
ALFABETO = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ23456789"

def generar_sal() -> str:
    return secrets.token_hex(LONGITUD_SAL)

def hashear(clave: str, sal: str) -> str:
    derivada = hashlib.pbkdf2_hmac(
        "sha256",
        clave.encode("utf-8"),
        sal.encode("utf-8"),
        ITERACIONES,
    )
    return derivada.hex()

def verificar(clave: str, sal: str, hash_esperado: str) -> bool:
    if not hash_esperado:
        return False
    return secrets.compare_digest(hashear(clave, sal), hash_esperado)

def generar_clave(longitud: int = 8) -> str:
    return "".join(secrets.choice(ALFABETO) for _ in range(longitud))

```

## 10.4 Almacenamiento en archivos JSON

**Ruta:** [sistema\\_academico/nucleo/almacen.py](#)

*Lee y escribe listas de diccionarios en archivos JSON. La escritura es atomica: primero escribe en un archivo temporal y despues lo renombra, para que un corte de luz no deje el JSON a medias.*

**105 lineas**

```

import json
import os
import tempfile
import threading
from pathlib import Path

class ArchivoInvalido(Exception):
    pass

class AlmacenJSON:
    def __init__(self, ruta, etiqueta: str = "registros"):
        self.ruta = Path(ruta)
        self.etiqueta = etiqueta
        self._candado = threading.Lock()
        self.ruta.parent.mkdir(parents=True, exist_ok=True)
        if not self.ruta.exists():
            self._escribir([])

    def _leer(self):
        try:
            contenido = self.ruta.read_text(encoding="utf-8").strip()
        except OSError as exc:
            raise ArchivoInvalido(f"No se pudo leer '{self.ruta.name}': {exc}") from exc
        if not contenido:
            return []
        try:
            datos = json.loads(contenido)
        except json.JSONDecodeError as exc:
            raise ArchivoInvalido(
                f"'{self.ruta.name}' esta danado. Corrigelo o eliminalo para regenerarlo."
            ) from exc
        if not isinstance(datos, list):
            raise ArchivoInvalido(f"'{self.ruta.name}' debe contener una lista.")
        return [item for item in datos if isinstance(item, dict)]

    def _escribir(self, registros):
        carpeta = self.ruta.parent
        temporal = None
        try:
            with tempfile.NamedTemporaryFile(
                "w",
                encoding="utf-8",
                dir=carpeta,
                delete=False,
                suffix=".tmp",
            ) as archivo:
                json.dump(registros, archivo, indent=2, ensure_ascii=False)
                temporal = archivo.name
            os.replace(temporal, self.ruta)
        except OSError as exc:
            if temporal and os.path.exists(temporal):
                os.unlink(temporal)
            raise ArchivoInvalido(f"No se pudo guardar '{self.ruta.name}': {exc}") from exc

    def listar(self):
        with self._candado:
            return self._leer()

```

```

def buscar_por_id(self, id_registro):
    for registro in self.listar():
        if registro.get("id") == id_registro:
            return registro
    return None

def guardar(self, registros):
    with self._candado:
        self._escribir(registros)

def agregar(self, registro):
    with self._candado:
        registros = self._leer()
        registros.append(registro)
        self._escribir(registros)
    return registro

def actualizar(self, id_registro, cambios):
    with self._candado:
        registros = self._leer()
        for indice, registro in enumerate(registros):
            if registro.get("id") == id_registro:
                actualizado = dict(registro)
                actualizado.update(cambios)
                registros[indice] = actualizado
        self._escribir(registros)
    return actualizado
    return None

def eliminar(self, id_registro):
    with self._candado:
        registros = self._leer()
        restantes = [r for r in registros if r.get("id") != id_registro]
        if len(restantes) == len(registros):
            return False
        self._escribir(restantes)
    return True

def eliminar_varios(self, ids):
    ids = set(ids)
    with self._candado:
        registros = self._leer()
        restantes = [r for r in registros if r.get("id") not in ids]
        self._escribir(restantes)
    return len(registros) - len(restantes)

```

## 10.5 Reglas de negocio y permisos

Ruta: [sistema\\_academico/nucleo/servicios.py](#)

*El corazon del programa. Valida los datos, controla que cada rol solo pueda hacer lo que le corresponde, genera los identificadores, calcula promedios y resuelve los borrados en cascada.*

463 lineas

```

from __future__ import annotations

import re
import unicodedata
from datetime import date

from config import (
    ARCHIVO_ESTUDIANTES,
    ARCHIVO_MATERIAS,
    ARCHIVO_NOTAS,
    ARCHIVO_USUARIOS,
    CREDITOS_MAXIMOS,
    CREDITOS_MINIMOS,
    NOTA_MAXIMA,
    NOTA_MINIMA,
)
from nucleo import seguridad
from nucleo.almacen import AlmacenJSON

ROLES = ("docente", "estudiante")
CAMPOS_ESTUDIANTE = ("cedula", "nombres", "apellidos", "email", "telefono",
CAMPOS_MATERIA = ("codigo", "nombre", "creditos", "semestre", "docente")
CAMPOS_NOTA = ("estudiante_id", "materia_id", "valor", "tipo", "periodo")

class ErrorValidacion(Exception):
    pass

class ErrorAutenticacion(Exception):
    pass

class ErrorPermiso(Exception):
    pass

def _sin_acentos(texto: str) -> str:
    normalizado = unicodedata.normalize("NFKD", texto)
    return "".join(c for c in normalizado if not unicodedata.combining(c))

def _normalizar(texto: str) -> str:
    return re.sub(r"\s+", " ", _sin_acentos(texto or "")).strip().lower()

def _validar_texto(valor, campo: str, obligatorio: bool = True, minimo=2, ma
    valor = (valor or "").strip()
    if not valor:
        if obligatorio:
            raise ErrorValidacion(f"El campo '{campo}' es obligatorio.")
        return ""
    if len(valor) < minimo:
        raise ErrorValidacion(f"'{campo}' debe tener al menos {minimo} caracteres.")
    if len(valor) > maximo:
        raise ErrorValidacion(f"'{campo}' no puede superar {maximo} caracteres.")
    return valor

def _validar_numero(valor, campo: str, minimo, maximo, decimales=0):

```

```

texto = str(valor if valor is not None else "").strip().replace(", ", ".")
if not texto:
    raise ErrorValidacion(f"El campo '{campo}' es obligatorio.")
try:
    numero = float(texto)
except ValueError as exc:
    raise ErrorValidacion(f"'{campo}' debe ser un numero valido.") from exc
if numero != numero or numero in (float("inf"), float("-inf")):
    raise ErrorValidacion(f"'{campo}' debe ser un numero valido.")
if not (minimo <= numero <= maximo):
    raise ErrorValidacion(f"'{campo}' debe estar entre {minimo} y {maximo}.")
return round(numero, decimales) if decimales else int(round(numero))

def _validar_correo(valor) -> str:
    valor = (valor or "").strip()
    if not valor:
        raise ErrorValidacion("El campo 'Email' es obligatorio.")
    if not re.fullmatch(r"^[^\s]+@[^\s]+\.[A-Za-z]{2,}$", valor):
        raise ErrorValidacion("El email no tiene un formato valido.")
    return valor

def _validar_cedula(valor) -> str:
    valor = re.sub(r"^[^0-9A-Za-z-]", "", (valor or "").strip().upper())
    if not valor:
        raise ErrorValidacion("El campo 'Cedula / Matricula' es obligatorio.")
    if len(valor) < 5 or len(valor) > 20:
        raise ErrorValidacion("La cedula debe tener entre 5 y 20 caracteres.")
    return valor

class Sesion:
    def __init__(self, usuario: dict):
        self.id = usuario["id"]
        self.nombre_usuario = usuario["nombre_usuario"]
        self.rol = usuario["rol"]
        self.nombre = usuario.get("nombre", usuario["nombre_usuario"])
        self.estudiante_id = usuario.get("estudiante_id")

    @property
    def es_docente(self) -> bool:
        return self.rol == "docente"

    @property
    def es_estudiante(self) -> bool:
        return self.rol == "estudiante"

    def __str__(self) -> str:
        return f"{self.nombre} ({self.rol})"

class Servicio:
    def __init__(self):
        self.usuarios = AlmacenJSON(ARCHIVO_USUARIOS, "usuarios")
        self.estudiantes = AlmacenJSON(ARCHIVO_ESTUDIANTES, "estudiantes")
        self.materias = AlmacenJSON(ARCHIVO_MATERIAS, "materias")
        self.notas = AlmacenJSON(ARCHIVO_NOTAS, "notas")
        self.sesion: Sesion | None = None
        self._asegurar_admin()

```

```

def _asegurar_admin(self):
if self.usuarios.listar():
return
self.usuarios.agregar(self._crear_usuario("admin", "admin123", "docente", "D

def _crear_usuario(self, nombre_usuario, clave, rol, nombre, estudiante_id):
sal = seguridad.generar_sal()
return {
"id": self._nuevo_id(self.usuarios.listar(), "USR"),
"nombre_usuario": nombre_usuario,
"sal": sal,
"clave_hash": seguridad.hashear(clave, sal),
"rol": rol,
"nombre": nombre,
"estudiante_id": estudiante_id,
}

@staticmethod
def _nuevo_id(registros, prefijo: str) -> str:
mayor = 0
for registro in registros:
partes = str(registro.get("id", "")).rsplit("-", 1)
if len(partes) == 2 and partes[1].isdigit():
mayor = max(mayor, int(partes[1]))
return f"{prefijo}-{mayor + 1:04d}"

def _exigir_docente(self):
if self.sesion is None:
raise ErrorPermiso("Debes iniciar sesion.")
if not self.sesion.es_docente:
raise ErrorPermiso("Solo los docentes pueden realizar esta accion.")

def _usuario_por_nombre(self, nombre_usuario):
objetivo = _normalizar(nombre_usuario)
for usuario in self.usuarios.listar():
if _normalizar(usuario.get("nombre_usuario", "")) == objetivo:
return usuario
return None

def _nombre_usuario_disponible(self, nombre_usuario, ignorar_id=None) -> str:
nombre_usuario = _validar_texto(nombre_usuario, "Usuario", minimo=3, maximo=
if not re.fullmatch(r"[A-Za-z0-9._-]+", nombre_usuario):
raise ErrorValidacion("El usuario solo admite letras, numeros, punto, guion
if self._usuario_por_nombre(nombre_usuario) is not None:
existente = self._usuario_por_nombre(nombre_usuario)
if ignorar_id is None or existente.get("id") != ignorar_id:
raise ErrorValidacion(f"El usuario '{nombre_usuario}' ya esta registrado.")
return nombre_usuario

def iniciar_sesion(self, nombre_usuario: str, clave: str) -> Sesion:
usuario = self._usuario_por_nombre(nombre_usuario)
if usuario is None:
raise ErrorAutenticacion("Usuario o contrasena incorrectos.")
if not seguridad.verificar(clave or "", usuario.get("sal", ""), usuario.get
raise ErrorAutenticacion("Usuario o contrasena incorrectos.")
self.sesion = Sesion(usuario)
return self.sesion

def cerrar_sesion(self):

```

```

self.sesion = None

def cambiar_contrasena(self, actual: str, nueva: str, confirmacion: str):
    if self.sesion is None:
        raise ErrorPermiso("Debes iniciar sesion.")
    usuario = self.usuarios.buscar_por_id(self.sesion.id)
    if usuario is None:
        raise ErrorAutenticacion("El usuario ya no existe.")
    if not seguridad.verificar(actual or "", usuario.get("sal", ""), usuario.get("clave_hash", "")):
        raise ErrorAutenticacion("La contrasena actual no es correcta.")
    if nueva != confirmacion:
        raise ErrorValidacion("La nueva contrasena no coincide con la confirmacion.")
    if len(nueva or "") < 4:
        raise ErrorValidacion("La nueva contrasena debe tener al menos 4 caracteres.")
    sal = seguridad.generar_sal()
    self.usuarios.actualizar(
        usuario["id"],
        {"sal": sal, "clave_hash": seguridad.hashar(nueva, sal)},
    )

def _sugerir_usuario(self, nombres: str, apellidos: str) -> str:
    partes = []
    for palabra in (nombres or "").split() + (apellidos or "").split():
        palabra = re.sub(r"[^A-Za-z0-9]", "", palabra)
        if palabra:
            partes.append(palabra[:2].lower())
    base = (".".join(partes[:3])) or "estudiante"
    candidato = base
    sufijo = 1
    while self._usuario_por_nombre(candidato) is not None:
        sufijo += 1
    candidato = f"{base}{sufijo}"
    return candidato

def cedula_ocupada(self, cedula, ignorar_id=None) -> bool:
    objetivo = cedula.upper()
    for estudiante in self.estudiantes.listar():
        if estudiante.get("cedula", "").upper() == objetivo and estudiante.get("id", "") != ignorar_id:
            return True
    return False

def agregar_estudiante(self, datos: dict) -> dict:
    self._exigir_docente()
    cedula = _validar_cedula(datos.get("cedula"))
    if self.cedula_ocupada(cedula):
        raise ErrorValidacion(f"Ya existe un estudiante con la cedula {cedula}.")
    nombres = _validar_texto(datos.get("nombres"), "Nombres", minimo=2, maximo=50)
    apellidos = _validar_texto(datos.get("apellidos"), "Apellidos", minimo=2, maximo=50)
    email = _validar_correo(datos.get("email"))
    telefono = _validar_texto(datos.get("telefono"), "Telefono", minimo=7, maximo=15)
    carrera = _validar_texto(datos.get("carrera"), "Carrera", minimo=3, maximo=50)

    usuario_solicitado = (datos.get("nombre_usuario") or "").strip()
    clave_solicitada = (datos.get("clave") or "").strip()
    if usuario_solicitado:
        nombre_usuario = self._nombre_usuario_disponible(usuario_solicitado)
    else:
        nombre_usuario = self._sugerir_usuario(nombres, apellidos)
    if clave_solicitada:
        if len(clave_solicitada) < 4:

```

```

raise ErrorValidacion("La contrasena debe tener al menos 4 caracteres.")
clave = clave_solicitada
else:
clave = seguridad.generar_clave()

estudiante = {
"id": self._nuevo_id(self.estudiantes.listar(), "EST"),
"cedula": cedula,
"nombres": nombres,
"apellidos": apellidos,
"email": email,
"telefono": telefono,
"carrera": carrera,
"activo": True,
"fecha_registro": date.today().isoformat(),
}
usuario = self._crear_usuario(
nombre_usuario, clave, "estudiante", f"{nombres} {apellidos}", estudiante["id"]
)
self.estudiantes.agregar(estudiante)
self.usuarios.agregar(usuario)
return dict(estudiante, nombre_usuario=nombre_usuario, clave=clave)

def actualizar_estudiante(self, id_estudiante: str, datos: dict) -> dict:
self._exigir_docente()
estudiante = self.estudiantes.buscar_por_id(id_estudiante)
if estudiante is None:
raise ErrorValidacion("El estudiante ya no existe.")
cedula = _validar_cedula(datos.get("cedula"))
if self.cedula_ocupada(cedula, ignorar_id=id_estudiante):
raise ErrorValidacion(f"Ya existe otro estudiante con la cedula {cedula}.")
cambios = {
"cedula": cedula,
"nombres": _validar_texto(datos.get("nombres"), "Nombres", minimo=2, maximo=30),
"apellidos": _validar_texto(datos.get("apellidos"), "Apellidos", minimo=2, maximo=30),
"email": _validar_correo(datos.get("email")),
"telefono": _validar_texto(datos.get("telefono"), "Telefono", minimo=7, maximo=15),
"carrera": _validar_texto(datos.get("carrera"), "Carrera", minimo=3, maximo=10)
}
actualizado = self.estudiantes.actualizar(id_estudiante, cambios)
usuario = self._usuario_del_estudiante(id_estudiante)
if usuario is not None:
self.usuarios.actualizar(
usuario["id"],
{"nombre": f"{cambios['nombres']} {cambios['apellidos']}"},
)
return actualizado

def eliminar_estudiante(self, id_estudiante: str):
self._exigir_docente()
usuario = self._usuario_del_estudiante(id_estudiante)
notas = [n for n in self.notas.listar() if n.get("estudiante_id") == id_estudiante]
if usuario is not None:
self.usuarios.eliminar(usuario["id"])
self.notas.eliminar_varios([n["id"] for n in notas])
self.estudiantes.eliminar(id_estudiante)
return len(notas)

def _usuario_del_estudiante(self, id_estudiante):
for usuario in self.usuarios.listar():

```

```

if usuario.get("estudiante_id") == id_estudiante:
    return usuario
return None

def restablecer_clave_estudiante(self, id_estudiante: str):
    self._exigir_docente()
    usuario = self._usuario_del_estudiante(id_estudiante)
    if usuario is None:
        raise ErrorValidacion("El estudiante no tiene un usuario asignado.")
    clave = seguridad.generar_clave()
    sal = seguridad.generar_sal()
    self.usuarios.actualizar(
        usuario["id"], {"sal": sal, "clave_hash": seguridad.hashear(clave, sal)}
    )
    return clave

def codigo_materia_ocupado(self, codigo, ignorar_id=None) -> bool:
    objetivo = _normalizar(codigo)
    for materia in self.materias.listar():
        if _normalizar(materia.get("codigo", "")) == objetivo and materia.get("id") != ignorar_id:
            return True
    return False

def agregar_materia(self, datos: dict) -> dict:
    self._exigir_docente()
    codigo = _validar_texto(datos.get("codigo"), "Codigo", minimo=3, maximo=15)
    if self.codigo_materia_ocupado(codigo):
        raise ErrorValidacion(f"Ya existe una materia con el codigo {codigo}.")
    materia = {
        "id": self._nuevo_id(self.materias.listar(), "MAT"),
        "codigo": codigo.upper(),
        "nombre": _validar_texto(datos.get("nombre"), "Nombre de la materia", minimo=3, maximo=15),
        "creditos": _validar_numero(datos.get("creditos"), "Creditos", CREDITOS_MINIMO, CREDITOS_MAXIMO),
        "semestre": _validar_numero(datos.get("semestre"), "Semestre", 1, 12),
        "docente": self.sesion.nombre if self.sesion else _validar_texto(datos.get("docente"), "Docente", minimo=3, maximo=15)
    }
    return self.materias.agregar(materia)

def actualizar_materia(self, id_materia: str, datos: dict) -> dict:
    self._exigir_docente()
    if self.materias.buscar_por_id(id_materia) is None:
        raise ErrorValidacion("La materia ya no existe.")
    codigo = _validar_texto(datos.get("codigo"), "Codigo", minimo=3, maximo=15)
    if self.codigo_materia_ocupado(codigo, ignorar_id=id_materia):
        raise ErrorValidacion(f"Ya existe otra materia con el codigo {codigo}.")
    cambios = {
        "codigo": codigo.upper(),
        "nombre": _validar_texto(datos.get("nombre"), "Nombre de la materia", minimo=3, maximo=15),
        "creditos": _validar_numero(datos.get("creditos"), "Creditos", CREDITOS_MINIMO, CREDITOS_MAXIMO),
        "semestre": _validar_numero(datos.get("semestre"), "Semestre", 1, 12),
    }
    return self.materias.actualizar(id_materia, cambios)

def eliminar_materia(self, id_materia: str):
    self._exigir_docente()
    notas = [n for n in self.notas.listar() if n.get("materia_id") == id_materia]
    self.notas.eliminar_varios([n["id"] for n in notas])
    self.materias.eliminar(id_materia)
    return len(notas)

```

```

def agregar_nota(self, datos: dict) -> dict:
    self._exigir_docente()
    estudiante = self.estudiantes.buscar_por_id(datos.get("estudiante_id"))
    if estudiante is None:
        raise ErrorValidacion("Selecciona un estudiante valido.")
    materia = self.materias.buscar_por_id(datos.get("materia_id"))
    if materia is None:
        raise ErrorValidacion("Selecciona una materia valida.")
    nota = {
        "id": self._nuevo_id(self.notas.listar(), "NOT"),
        "estudiante_id": estudiante["id"],
        "materia_id": materia["id"],
        "valor": _validar_numero(datos.get("valor"), "Nota", NOTA_MINIMA, NOTA_MAXIMA),
        "tipo": _validar_texto(datos.get("tipo"), "Tipo de nota", minimo=3, maximo=3),
        "periodo": _validar_texto(datos.get("periodo"), "Periodo", minimo=4, maximo=4),
        "fecha": date.today().isoformat(),
    }
    return self.notas.agregar(nota)

def actualizar_nota(self, id_nota: str, datos: dict) -> dict:
    self._exigir_docente()
    nota = self.notas.buscar_por_id(id_nota)
    if nota is None:
        raise ErrorValidacion("La nota ya no existe.")
    cambios = {
        "valor": _validar_numero(datos.get("valor"), "Nota", NOTA_MINIMA, NOTA_MAXIMA),
        "tipo": _validar_texto(datos.get("tipo"), "Tipo de nota", minimo=3, maximo=3),
        "periodo": _validar_texto(datos.get("periodo"), "Periodo", minimo=4, maximo=4),
    }
    if datos.get("estudiante_id"):
        if self.estudiantes.buscar_por_id(datos["estudiante_id"]) is None:
            raise ErrorValidacion("Selecciona un estudiante valido.")
        cambios["estudiante_id"] = datos["estudiante_id"]
    if datos.get("materia_id"):
        if self.materias.buscar_por_id(datos["materia_id"]) is None:
            raise ErrorValidacion("Selecciona una materia valida.")
        cambios["materia_id"] = datos["materia_id"]
    return self.notas.actualizar(id_nota, cambios)

def eliminar_nota(self, id_nota: str) -> bool:
    self._exigir_docente()
    return self.notas.eliminar(id_nota)

def nombre_estudiante(self, id_estudiante) -> str:
    estudiante = self.estudiantes.buscar_por_id(id_estudiante)
    if estudiante is None:
        return "(eliminado)"
    return f"{estudiante['nombres']} {estudiante['apellidos']}"

def materia_por_id(self, id_materia):
    return self.materias.buscar_por_id(id_materia)

def codigo_materia(self, id_materia) -> str:
    materia = self.materias.buscar_por_id(id_materia)
    return materia["codigo"] if materia else "(eliminada)"

def nombre_materia(self, id_materia) -> str:
    materia = self.materias.buscar_por_id(id_materia)
    return materia["nombre"] if materia else "(materia eliminada)"

```

```

def notas_de(self, id_estudiante):
    registros = [n for n in self.notas.listar() if n.get("estudiante_id") == id]
    return sorted(registros, key=lambda n: (n.get("periodo", ""), n.get("materia")))

def resumen_estudiante(self, id_estudiante) -> dict:
    notas = self.notas_de(id_estudiante)
    if not notas:
        return {"promedio": 0.0, "aprobadas": 0, "reprobadas": 0, "total": 0}
    valores = [n["valor"] for n in notas]
    return {
        "promedio": round(sum(valores) / len(valores), 2),
        "aprobadas": sum(1 for v in valores if v >= 60),
        "reprobadas": sum(1 for v in valores if v < 60),
        "total": len(valores),
    }

def promedios_por_materia(self, id_estudiante):
    agrupado = {}
    for nota in self.notas_de(id_estudiante):
        agrupado.setdefault(nota["materia_id"], []).append(nota["valor"])
    return {
        id_materia: {
            "promedio": round(sum(valores) / len(valores), 2),
            "total": len(valores),
        }
        for id_materia, valores in agrupado.items()
    }

def indice_por_id(self, almacen, clave="id"):
    return {registro.get(clave): registro for registro in almacen.listar()}

def estadisticas_generales(self) -> dict:
    estudiantes = self.estudiantes.listar()
    notas = self.notas.listar()
    valores = [n["valor"] for n in notas]
    return {
        "estudiantes": len(estudiantes),
        "materias": len(self.materias.listar()),
        "notas": len(notas),
        "promedio_general": round(sum(valores) / len(valores), 2) if valores else 0,
        "aprobadas": sum(1 for v in valores if v >= 60),
        "reprobadas": sum(1 for v in valores if v < 60),
    }

```

## 10.6 Paquete interfaz (vacío)

Ruta: [sistema\\_academico/interfaz/\\_\\_init\\_\\_.py](#)

Marca la carpeta como paquete de Python. No lleva código.

(archivo vacío de propósito)

## 10.7 Tema visual

Ruta: [sistema\\_academico/interfaz/estilos.py](#)

Define los colores y los estilos de ttk (botones, tablas, etiquetas, pestañas). Todo el aspecto de la aplicación sale de aquí.

69 líneas

```

import tkinter as tk
from tkinter import ttk

AZUL = "#1f4e79"
AZUL_CLARO = "#2e6da4"
AZUL_SUAVE = "#e8f0f8"
GRIS_FONDO = "#f4f6f8"
GRIS_TEXTO = "#555555"
VERDE = "#1e7e34"
ROJO = "#c0392b"
AMBAR = "#b9770e"

def aplicar_estilos(root: tk.Misc):
    try:
        ttk.Style().theme_use("vista")
    except tk.TclError:
        try:
            ttk.Style().theme_use("clam")
        except tk.TclError:
            pass

    estilo = ttk.Style()
    estilo.configure(".", font=("Segoe UI", 10))

    estilo.configure("TFrame", background=GRIS_FONDO)
    estilo.configure("Superior.TFrame", background=AZUL)
    estilo.configure("Superior.TLabel", background=AZUL, foreground="white", font=VERDE)
    estilo.configure("SuperiorRol.TLabel", background=AZUL, foreground="#cfe2f3", font=VERDE)

    estilo.configure("TLabel", background=GRIS_FONDO, foreground=GRIS_TEXTO)
    estilo.configure("Campo.TLabel", background="ffffff", foreground=GRIS_TEXTO)
    estilo.configure("Negrita.TLabel", background=GRIS_FONDO, foreground="#222222", font=VERDE)
    estilo.configure("Error.TLabel", background=GRIS_FONDO, foreground=ROJO, font=VERDE)
    estilo.configure("Aviso.TLabel", background=GRIS_FONDO, foreground=VERDE, font=VERDE)
    estilo.configure("Seccion.TLabel", background=GRIS_FONDO, foreground=AZUL, font=VERDE)

    estilo.configure("Tarjeta.TLabelframe", background=GRIS_FONDO, borderwidth=3)
    estilo.configure("Tarjeta.TLabelframe.Label", background=GRIS_FONDO, foreground=VERDE, font=VERDE)

    estilo.configure("TButton", padding=(12, 6), font=("Segoe UI", 10))
    estilo.configure("Primario.TButton", padding=(12, 6), font=("Segoe UI", 10), foreground=VERDE)
    estilo.configure("Peligro.TButton", padding=(10, 6), font=("Segoe UI", 9), foreground=AMBAR)

    estilo.configure("TLabelframe", background=GRIS_FONDO)
    estilo.configure("TLabelframe.Label", background=GRIS_FONDO, foreground=AZUL, font=VERDE)

    estilo.configure("Accion.TButton", font=("Segoe UI", 9), padding=(8, 4))
    estilo.configure("Buscador.TEntry", padding=4)

    estilo.configure("Estadistica.TLabelframe", background="ffffff", relief="solid")
    estilo.configure("Estadistica.TLabelframe.Label", background=GRIS_FONDO, foreground=VERDE, font=VERDE)
    estilo.configure("Valor.TLabel", background=GRIS_FONDO, foreground=AZUL, font=VERDE)

    estilo.configure("Treeview", rowheight=24, font=("Segoe UI", 9))
    estilo.configure("Treeview.Heading", font=("Segoe UI", 9, "bold"))
    estilo.map("Treeview", background=[("selected", AZUL_CLARO)], foreground=[("selected", VERDE)])

    estilo.configure("TNotebook", background=GRIS_FONDO, borderwidth=0)
    estilo.configure("TNotebook.Tab", padding=(16, 8), font=("Segoe UI", 10))

    estilo.map("TNotebook.Tab", background=[("selected", "ffffff")], foreground=[("selected", VERDE)])

    estilo.configure("TCheckbutton", background="ffffff")
    estilo.configure("Blanco.TCheckbutton", background="ffffff")
    estilo.configure("Campo.TEntry", padding=4)
    estilo.configure("Campo.TCombobox", padding=4)

    root.option_add("*tearOff", False)
    return estilo

```

## 10.8 Componentes reutilizables

Ruta: [sistema\\_academico/interfaz/widgets.py](#)

*Clases que se usan en todos los formularios: CampoTexto, CampoSeleccion, Tabla y TarjetaEstadistica. Evita repetir codigo en cada pantalla.*

157 lineas

```
import tkinter as tk
from tkinter import ttk

from interfaz import estilos

class Aviso:
    def __init__(self, master, texto_inicial=""):
        self.var = tk.StringVar(value=texto_inicial)
        self.label = ttk.Label(master, textvariable=self.var, style="Aviso.TLabel",

    def exito(self, texto):
        self.var.set(texto)
        self.label.configure(style="Aviso.TLabel")

    def error(self, texto):
        self.var.set(texto)
        self.label.configure(style="Error.TLabel")

    def info(self, texto):
        self.var.set(texto)
        self.label.configure(style="Aviso.TLabel")

    def limpiar(self):
        self.var.set("")

class CampoTexto:
    def __init__(self, master, etiqueta, valor="", ancho=34, tipo="texto", ayuda=""):
        self.tipo = tipo
        self.var = tk.StringVar(value=str(valor))
        self.unidad = unidad

        marco = tk.Frame(master, bg="ffffff", highlightbackground="#d0d7de", highlightthickness=1)
        marco.pack(fill="x", padx=18, pady=5)

        fila = tk.Frame(marco, bg="ffffff")
        fila.pack(fill="x", padx=10, pady=(7, 2))

        ttk.Label(fila, text=etiqueta + " ", style="Campo.TLabel").pack(side="left")
        if unidad:
            ttk.Label(fila, text=unidad, style="Campo.TLabel").pack(side="right")

        self.entrada = ttk.Entry(
            marco,
            textvariable=self.var,
            width=ancho,
            style="Campo.TEntry",
            show="*" if tipo == "clave" else "",
            justify="left",
        )
        self.entrada.pack(fill="x", padx=10, pady=(0, 8))

        if ayuda:
            ttk.Label(marco, text=ayuda, style="Campo.TLabel", foreground="#8a8a8a").pack(
                side="bottom",
                anchor="w",
                padx=10,
                pady=(0, 6)
            )

    def obtener(self):
        return self.var.get().strip()
```

```

def fijar(self, valor):
    self.var.set(str(valor))

def enfocar(self):
    self.entrada.focus_set()
    self.entrada.selection_range(0, "end")

class CampoSeleccion:
    def __init__(self, master, etiqueta, valores, ancho=32, ayuda=""):
        self.var = tk.StringVar()

        marco = tk.Frame(master, bg="ffffff", highlightbackground="#d0d7de", highlightthickness=1)
        marco.pack(fill="x", padx=18, pady=5)

        ttk.Label(marco, text=etiqueta + " **", style="Campo.TLabel").pack(anchor="w", fill="x",
            expand=True, ipadx=10, ipady=5)

        combo = ttk.Combobox(
            marco,
            textvariable=self.var,
            values=list(valores),
            width=ancho,
            state="readonly",
            style="Campo.TCombobox",
        )
        combo.pack(fill="x", padx=10, pady=(0, 8))

        if ayuda:
            ttk.Label(marco, text=ayuda, style="Campo.TLabel", foreground="#8a8a8a").pack(
                anchor="w", padx=10, pady=(0, 6)
            )
        self.combo = combo

    def obtener(self):
        return self.var.get()

    def fijar(self, valor):
        if valor:
            self.var.set(valor)

    def habilitar(self, estado):
        self.combo.configure(state="readonly" if estado else "disabled")

class Tabla:
    def __init__(self, master, columnas, titulos, anchos, altura=12, on_doble_click=None):
        self.marco = ttk.Frame(master)
        self.marco.pack(fill="both", expand=True, padx=12, pady=(8, 4))

        self.arbol = ttk.Treeview(
            self.marco,
            columns=columnas,
            show="headings",
            height=altura,
            selectmode="browse",
        )
        for column, titulo, ancho in zip(columnas, titulos, anchos):
            self.arbol.heading(column, text=titulo)
            self.arbol.column(column, width=ancho, anchor="w", stretch=True)

```

```

barra = ttk.Scrollbar(self.marco, orient="vertical", command=self.arbol.yview)
self.arbol.configure(yscrollcommand=barra.set)
self.arbol.pack(side="left", fill="both", expand=True)
barra.pack(side="right", fill="y")

if on_doble_clic:
self.arbol.bind("<Double-1>", lambda _e: on_doble_clic(self.id_seleccionada))
if on_seleccion:
self.arbol.bind("<<TreeviewSelect>>", lambda _e: on_seleccion(self.id_seleccionada))

def limpiar(self):
self.arbol.delete(*self.arbol.get_children())

def insertar(self, id_registro, valores, etiqueta=""):
self.arbol.insert("", "end", iid=str(id_registro), values=valores, tags=(etiqueta,))

def id_seleccionada(self):
seleccion = self.arbol.selection()
return seleccion[0] if seleccion else None

def resultado_si_existe(self, id_registro):
if id_registro and self.arbol.exists(str(id_registro)):
self.arbol.selection_set(str(id_registro))
self.arbol.focus(str(id_registro))
self.arbol.see(str(id_registro))

class TarjetaEstadistica:
def __init__(self, master, titulo, valor_inicial="0"):
marco = ttk.LabelFrame(master, text=titulo, style="Estadistica.TLabelFrame")
self.var = tk.StringVar(value=str(valor_inicial))
ttk.Label(marco, textvariable=self.var, style="Valor.TLabel").pack(anchor="w")
marco.pack(side="left", fill="x", expand=True, padx=5, pady=(10, 0))

def actualizar(self, valor):
self.var.set(str(valor))

```

## 10.9 Pantalla de inicio de sesion

Ruta: `sistema_academico/interfaz/vista_login.py`

*Tarjeta centrada con el formulario de usuario y contrasena. Valida las credenciales y decide que panel abrir segun el rol.*

141 lineas

```

import tkinter as tk
from tkinter import ttk

from config import TITULO_APP, VERSION
from interfaz import estilos, widgets
from nucleo.servicios import ErrorAutenticacion

class VistaLogin(ttk.Frame):
    ANCHO_MINIMO = 440

    def __init__(self, master, servicio, al_entrar):
        super().__init__(master, style="TFrame")
        self.servicio = servicio
        self.al_entrar = al_entrar

    tarjeta = tk.Frame(
        self,
        bg="ffffff",
        highlightbackground="#d7dee5",
        highlightthickness=1,
    )
    self.tarjeta = tarjeta

    cabecera = tk.Frame(tarjeta, bg=estilos.AZUL, height=76)
    cabecera.pack(fill="x")
    cabecera.pack_propagate(False)

    tk.Label(
        cabecera,
        text=TITULO_APP,
        bg=estilos.AZUL,
        fg="white",
        font=("Segoe UI", 16, "bold"),
    ).pack(pady=(16, 0))
    tk.Label(
        cabecera,
        text=f"Gestion de estudiantes, materias y notas - v{VERSION}",
        bg=estilos.AZUL,
        fg="#cfe2f3",
        font=("Segoe UI", 8),
    ).pack(pady=(2, 16))

    cuerpo = tk.Frame(tarjeta, bg="ffffff")
    cuerpo.pack(fill="both", expand=True, padx=12, pady=(6, 4))

    tk.Label(
        cuerpo,
        text="Iniciar sesion",
        bg="ffffff",
        fg=estilos.AZUL,
        font=("Segoe UI", 12, "bold"),
    ).pack(anchor="w", padx=8, pady=(6, 2))

    self.campo_usuario = widgets.CampoTexto(cuerpo, "Usuario", ancho=36)
    self.campo_clave = widgets.CampoTexto(cuerpo, "Contraseña", tipo="clave", an

    opciones = tk.Frame(cuerpo, bg="ffffff")
    opciones.pack(fill="x", padx=28, pady=(2, 4))
    self.ver_clave = tk.BooleanVar(value=False)

```

```

ttk.Checkbutton(
opciones,
text="Mostrar contrasena",
variable=self.ver_clave,
command=self._alternar_clave,
style="Blanco.TCheckbutton",
).pack(side="left")

self.credenciales = tk.StringVar(value="Usuario por defecto: admin / admin123")
tk.Label(
cuerpo,
textvariable=self.credenciales,
bg="ffffff",
fg="#8a8a8a",
font=("Segoe UI", 8),
wraplength=390,
justify="left",
).pack(anchor="w", padx=28, pady=(4, 8))

self.avisos = widgets.Aviso(cuerpo)
self.avisos.pack(anchor="w", padx=28, pady=(0, 6))

botones = tk.Frame(cuerpo, bg="ffffff")
botones.pack(fill="x", padx=28, pady=(2, 12))
tk.Button(botones, text="Entrar", command=self._entrar, style="Primario.TButton",
side="left"
)
tk.Button(botones, text="Salir", command=self._salir, width=10).pack(side="right")

for widget in (self.campo_usuario, self.campo_clave):
widget.entrada.bind("<Return>", lambda _e: self._entrar())
self.campo_usuario.enfocar()

self._ajustar()
self.bind("<Configure>", self._al_redimensionar)

def _ajustar(self):
self.update_idletasks()
ancho = max(self.ANCHO_MINIMO, self.tarjeta.winfo_reqwidth())
alto = self.tarjeta.winfo_reqheight()
disponible_ancho = max(ancho, self.winfo_width())
disponible_alto = max(alto, self.winfo_height())
x = max(0, (disponible_ancho - ancho) // 2)
y = max(0, (disponible_alto - alto) // 2)
self.tarjeta.place(x=x, y=y, width=ancho, height=alto)
self._ultimo_tamano = (self.winfo_width(), self.winfo_height())

def _al_redimensionar(self, _evento):
actual = (self.winfo_width(), self.winfo_height())
if actual != getattr(self, "_ultimo_tamano", None):
self._ajustar()

def _alternar_clave(self):
self.campo_clave.entrada.configure(show="" if self.ver_clave.get() else "*")

def _entrar(self):
usuario = self.campo_usuario.obtener()
clave = self.campo_clave.obtener()
if not usuario:
self.avisos.error("Escribe tu nombre de usuario.")

```

```
self.campo_usuario.enfocar()
return
if not clave:
self.aviso.error("Escribe tu contrasena.")
self.campo_clave.enfocar()
return
try:
sesion = self.servicio.iniciar_sesion(usuario, clave)
except ErrorAutenticacion as error:
self.aviso.error(str(error))
self.campo_clave.fijar("")
self.campo_clave.enfocar()
return
self.aviso.limpiar()
self.al_entrar(sesion)

def _salir(self):
self.campo_usuario.fijar("")
self.campo_clave.fijar("")
self.aviso.limpiar()
self.campo_usuario.enfocar()
```

## 10.10 Formularios (ventanas emergent)

Ruta: [sistema\\_academico/interfaz/dialogos.py](#)

*Las cuatro ventanas de formulario: estudiante, materia, nota y cambio de contrasena. Contienen la logica de alto automatico y la barra de scroll.*

501 lineas

```

import tkinter as tk
from tkinter import messagebox, ttk

from config import CREDITOS_MAXIMOS, CREDITOS_MINIMOS, NOTA_MAXIMA, NOTA_MINIMA
from interfaz import estilos, widgets
from nucleo.servicios import ErrorAutenticacion, ErrorPermiso, ErrorValidacion

MARGEN_PANTALLA = 80
BORDE_HORIZONTAL = 28
BORDE_VERTICAL = 30

class AreaDesplazable(tk.Frame):
    def __init__(self, master):
        super().__init__(master, bg=estilos.GRIS_FONDO)
        self.lienzo = tk.Canvas(self, highlightthickness=0, bd=0, bg=estilos.GRIS_FONDO)
        self.barra = ttk.Scrollbar(self, orient="vertical", command=self.lienzo.yview)
        self.lienzo.configure(yscrollcommand=self.barra.set)
        self.lienzo.pack(side="left", fill="both", expand=True)
        self.interior = tk.Frame(self.lienzo, bg=estilos.GRIS_FONDO)
        self.ventana = self.lienzo.create_window((0, 0), window=self.interior, anchor="nw")
        self.interior.bind("<Configure>", self._al_redimensionar_interior)
        self.lienzo.bind("<Configure>", self._al_redimensionar_lienzo)

    def _al_redimensionar_interior(self, _evento):
        self.lienzo.configure(scrollregion=self.lienzo.bbox("all"))

    def _al_redimensionar_lienzo(self, evento):
        self.lienzo.itemconfigure(self.ventana, width=evento.width)

    def desplazar(self, delta):
        self.lienzo.yview_scroll(delta, "units")

    def paginas(self):
        try:
            low, high = self.lienzo.yview()
        except tk.TclError:
            return 0
        return high - low

class DialogoBase(tk.Toplevel):
    def __init__(self, master, titulo, ancho=560, alto=None):
        super().__init__(master)
        self.title(titulo)
        self.resizable(False, False)
        self.configure(bg=estilos.GRIS_FONDO)
        self.transient(master)
        self.valores = None
        self.ancho, self.alto = ancho, alto

        self.pie = tk.Frame(self, bg=estilos.GRIS_FONDO)
        self.pie.pack(side="bottom", fill="x", padx=14, pady=12)

        self.area = AreaDesplazable(self)
        self.area.pack(fill="both", expand=True, padx=14, pady=(14, 0))

        self.cuerpo = tk.Frame(
            self.area.interior, bg="ffffff", highlightbackground="#d7dee5", highlightthickness=1

```

```

)
self.cuerpo.pack(fill="both", expand=True)

self.avisos = widgets.Avisos(self.pie)
self.avisos.label.pack(anchor="w", padx=6)

acciones = tk.Frame(self.pie, bg=estilos.GRIS_FONDO)
acciones.pack(side="right", anchor="e")
tk.Button(acciones, text="Cancelar", command=self.cancelar, width=12).pack(
tk.Button(acciones, text="Guardar", command=self.guardar, style="Primario",
side="right"
)

self._rueda = self._al_girar_rueda
self.bind_all("<MouseWheel>", self._rueda)
self.bind_all("<Button-4>", self._rueda)
self.bind_all("<Button-5>", self._rueda)
self.protocol("WM_DELETE_WINDOW", self.cancelar)

def _al_girar_rueda(self, evento):
if getattr(self, "_destruido", False) or not self.area.paginas() > 0:
return
delta = -1 if getattr(evento, "num", 0) == 4 or evento.delta > 0 else 1
self.area.desplazar(delta)
return "break"

def destroy(self):
if getattr(self, "_destruido", False):
return
self._destruido = True
for secuencia in ("<MouseWheel>", "<Button-4>", "<Button-5>"):
try:
self.unbind_all(secuencia)
except tk.TclError:
pass
super().destroy()

def ajustar(self):
self.update_idletasks()
ancho = max(int(self.ancho or 560), self.area.interior.winfo_reqwidth() + BORDE_VERTICAL)
alto_pie = self.pie.winfo_reqheight()
alto_texto = self.area.interior.winfo_reqheight()
alto = min(alto_texto + alto_pie + BORDE_VERTICAL, max(300, self.winfo_screenheight() - alto_pie))
if alto < alto_texto + alto_pie + BORDE_VERTICAL:
self.area.barra.pack(side="right", fill="y")
else:
self.area.barra.pack_forget()

x = self.master.winfo_rootx() + max(0, (self.master.winfo_width() - ancho) // 2)
y = self.master.winfo_rooty() + max(0, (self.master.winfo_height() - alto) // 2)
x = min(max(0, x), max(0, self.winfo_screenwidth() - ancho))
y = min(max(0, y), max(0, self.winfo_screenheight() - alto))
self.geometry(f"{ancho}x{alto}+{x}+{y}")

def titulo(self, texto, subtítulo=""):
marco = tk.Frame(self.cuerpo, bg="#ffffff")
marco.pack(fill="x", padx=18, pady=(14, 6))
tk.Label(marco, text=texto, bg="#ffffff", fg=estilos.AZUL, font=("Segoe UI", 14))
if subtítulo:
tk.Label(

```

```

marco,
text=Subtitulo,
bg="#ffffff",
fg=estilos.GRIS_TEXTO,
font=("Segoe UI", 9),
wraplength=ancho_fijo(self) - 60,
justify="left",
).pack(anchor="w", pady=(2, 0))

def aceptar(self):
    self.ajustar()
    self.grab_set()
    self.focus_set()

def guardar(self):
    raise NotImplementedError

def cancelar(self):
    self.valores = None
    self.destroy()

def mostrar(self):
    self.wait_window()
    return self.valores

def ancho_fijo(dialogo):
    return int(dialogo.ancha) if dialogo.ancha else 520

class DialogoEstudiante(DialogoBase):
    def __init__(self, master, servicio, estudiante=None):
        super().__init__(master, "Estudiante", ancho=540)
        self.servicio = servicio
        self.estudiante = estudiante
        self.editando = estudiante is not None

    if self.editando:
        self.titulo(
            "Editar estudiante",
            "Modifica los datos del alumno. La cedula debe seguir siendo unica en el sis
        )
    else:
        self.titulo(
            "Nuevo estudiante",
            "Registra un alumno y su usuario de acceso al sistema. Si dejas vacios el us
            "contrasena, se generan automaticamente.",
        )

    marco = tk.Frame(self.cuerpo, bg="#ffffff")
    marco.pack(fill="both", expand=True)

    self.cedula = widgets.CampoTexto(marco, "Cedula / Matricula", ancho=40)
    self.nombres = widgets.CampoTexto(marco, "Nombres", ancho=40)
    self.apellidos = widgets.CampoTexto(marco, "Apellidos", ancho=40)
    self.email = widgets.CampoTexto(marco, "Email", ancho=40)
    self.telefono = widgets.CampoTexto(marco, "Telefono", ancho=40)
    self.carrera = widgets.CampoTexto(marco, "Carrera", ancho=40)

    separador = tk.Frame(marco, bg="#e3e8ee", height=1)

```

```

separador.pack(fill="x", padx=18, pady=8)

self.usuario = widgets.CampoTexto(
    marco,
    "Usuario de acceso",
    ancho=24,
    ayuda="Solo si dejas el campo vacio el sistema lo arma automaticamente.",
)
self.clave = widgets.CampoTexto(
    marco,
    "Contrasena inicial",
    tipo="clave",
    ancho=24,
    ayuda="Minimo 4 caracteres. Si la dejas vacia se genera una y se muestra al"
)

if self.editando:
    for campo in (self.usuario, self.clave):
        campo.entrada.configure(state="disabled")
    tk.Label(
        marco,
        text="Las credenciales solo se cambian desde 'Restablecer contrasena'.",
        bg="ffffff",
        fg="#8a8a8a",
        font=("Segoe UI", 8),
    ).pack(anchor="w", padx=28, pady=(0, 10))
else:
    tk.Label(
        marco,
        text="Deja vacio lo que quieras autogenerar.",
        bg="ffffff",
        fg="#8a8a8a",
        font=("Segoe UI", 8),
    ).pack(anchor="w", padx=28, pady=(0, 10))

if self.editando:
    self._cargar(estudiante)

for campo in (self.cedula, self.nombres, self.apellidos, self.email, self.telefono):
    campo.entrada.bind("<Return>", lambda _e: self.guardar())
if not self.editando:
    for campo in (self.usuario, self.clave):
        campo.entrada.bind("<Return>", lambda _e: self.guardar())

self.cedula.enfocar()
self.aceptar()

def _cargar(self, estudiante):
    self.cedula.fijar(estudiante.get("cedula", ""))
    self.nombres.fijar(estudiante.get("nombres", ""))
    self.apellidos.fijar(estudiante.get("apellidos", ""))
    self.email.fijar(estudiante.get("email", ""))
    self.telefono.fijar(estudiante.get("telefono", ""))
    self.carrera.fijar(estudiante.get("carrera", ""))

def guardar(self):
    datos = {
        "cedula": self.cedula.obtener(),
        "nombres": self.nombres.obtener(),
        "apellidos": self.apellidos.obtener(),
    }

```

```

"email": self.email.obtener(),
"telefono": self.telefono.obtener(),
"carrera": self.carrera.obtener(),
"nombre_usuario": self.usuario.obtener(),
"clave": self.clave.obtener(),
}
try:
if self.editando:
resultado = self.servicio.actualizar_estudiante(self.estudiante["id"], datos)
mensaje = f"Datos de {resultado['nombres']} {resultado['apellidos']} actualizados"
else:
resultado = self.servicio.agregar_estudiante(datos)
mensaje = (
f"Estudiante creado.\n\nUsuario: {resultado['nombre_usuario']}\n"
f"Contraseña: {resultado['clave']}\n\nAnotala, no volvera a mostrarse."
)
except ErrorValidacion as error:
self.avisos.error(str(error))
return

self.valores = {"id": resultado["id"], "mensaje": mensaje}
self.destroy()
if not self.editando:
messagebox.showinfo(title="Estudiante registrado", message=mensaje, parent=self)

class DialogoMateria(DialogoBase):
def __init__(self, master, servicio, materia=None):
super().__init__(master, "Materia", ancho=520)
self.servicio = servicio
self.materia = materia
self.editando = materia is not None

self.titulo(
"Editar materia" if self.editando else "Nueva materia",
"Define el codigo, los creditos y el semestre en el que se imparte.",
)

marco = tk.Frame(self.cuerpo, bg="#ffffff")
marco.pack(fill="both", expand=True)

self.codigo = widgets.CampoTexto(
marco,
"Codigo",
ancho=24,
ayuda=f"Entre 3 y 15 caracteres, por ejemplo MAT101.",
)
self.nombre = widgets.CampoTexto(marco, "Nombre de la materia", ancho=40)
self.creditos = widgets.CampoTexto(
marco,
"Creditos",
ancho=12,
unidad=f"({CREDITOS_MINIMOS} - {CREDITOS_MAXIMOS})",
)
self.semestre = widgets.CampoTexto(marco, "Semestre", ancho=12, unidad="(1 - 4)")

separador = tk.Frame(marco, bg="#e3e8ee", height=1)
separador.pack(fill="x", padx=18, pady=10)

self.docente = tk.StringVar(value=servicio.sesion.nombre if servicio.sesion

```

```

marco_docente = tk.Frame(marco, bg="ffffff")
marco_docente.pack(fill="x", padx=18, pady=5)
ttk.Label(marco_docente, text="Docente responsable **", style="Campo.TLabel",
tk.Entry(
marco_docente,
textvariable=self.docente,
state="disabled",
style="Campo.TEntry",
width=40,
).pack(fill="x", pady=(4, 8))

tk.Label(
marco,
text="El docente se toma de la sesion iniciada.",
bg="ffffff",
fg="#8a8a8a",
font=("Segoe UI", 8),
).pack(anchor="w", padx=28, pady=(0, 10))

if self.editando:
self.codigo.fijar(materia.get("codigo", ""))
self.nombre.fijar(materia.get("nombre", ""))
self.creditos.fijar(materia.get("creditos", ""))
self.semestre.fijar(materia.get("semestre", ""))
self.docente.set(materia.get("docente", ""))

for campo in (self.codigo, self.nombre, self.creditos, self.semestre):
campo.entrada.bind("<Return>", lambda _e: self.guardar())

self.codigo.enfocar()
self.aceptar()

def guardar(self):
datos = {
"codigo": self.codigo.obtener(),
"nombre": self.nombre.obtener(),
"creditos": self.creditos.obtener(),
"semestre": self.semestre.obtener(),
"docente": self.docente.get(),
}
try:
if self.editando:
resultado = self.servicio.actualizar_materia(self.materia["id"], datos)
mensaje = f"Materia {resultado['codigo']} actualizada."
else:
resultado = self.servicio.agregar_materia(datos)
mensaje = f"Materia {resultado['codigo']} registrada."
except ErrorValidacion as error:
self.aviso.error(str(error))
return
self.valores = {"id": resultado["id"], "mensaje": mensaje}
self.destroy()

class DialogoNota(DialogoBase):
def __init__(self, master, servicio, nota=None, estudiante_id=None, materia=None):
super().__init__(master, "Nota", ancho=540)
self.servicio = servicio
self.nota = nota
self.editando = nota is not None

```

```

self.titulo(
"Editar nota" if self.editando else "Registrar nota",
f"Las notas van de {NOTA_MINIMA:g} a {NOTA_MAXIMA:g} puntos. El aprobado es
")

marco = tk.Frame(self.cuerpo, bg="#ffffff")
marco.pack(fill="both", expand=True)

estudiantes = self.servicio.estudiantes.listar()
materias = self.servicio.materias.listar()

if not estudiantes:
self.aviso.error("Primero registra al menos un estudiante.")
marco.pack_forget()
elif not materias:
self.aviso.error("Primero registra al menos una materia.")
marco.pack_forget()

self.etiquetas_estudiante = {
f"{e['nombres']} {e['apellidos']} ({e['cedula']}): e['id'] for e in estudiantes
}
self.etiquetas_materia = {f"{m['codigo']} - {m['nombre']}": m["id"] for m in materias}

self.estudiante = widgets.CampoSeleccion(marco, "Estudiante", list(self.etiquetas_estudiante.items()))
self.materia = widgets.CampoSeleccion(marco, "Materia", list(self.etiquetas_materia.items()))
self.valor = widgets.CampoTexto(
marco,
"Nota",
ancho=14,
unidad=f"({NOTA_MINIMA:g} - {NOTA_MAXIMA:g})",
)
self.tipo = widgets.CampoTexto(marco, "Tipo de nota", valor="Parcial", ancho=20)
self.periodo = widgets.CampoTexto(marco, "Periodo", valor="2026-1", ancho=20)

if not self.editando:
if estudiante_id:
for etiqueta, id_ in self.etiquetas_estudiante.items():
if id_ == estudiante_id:
self.estudiante.fijar(etiqueta)
break
if materia_id:
for etiqueta, id_ in self.etiquetas_materia.items():
if id_ == materia_id:
self.materia.fijar(etiqueta)
break

if self.editando:
self._cargar(nota, estudiantes, materias)
self.estudiante.habilitar(False)
self.materia.habilitar(False)

for campo in (self.valor, self.tipo, self.periodo):
campo.entrada.bind("<Return>", lambda _e: self.guardar())

if self.editando:
self.valor.enfocar()
else:
self.estudiante.combo.focus_set()
self.aceptar()

```

```

def _cargar(self, nota, estudiantes, materias):
    for estudiante in estudiantes:
        if estudiante["id"] == nota.get("estudiante_id"):
            self.estudiante.fijar(f"{estudiante['nombres']} {estudiante['apellidos']}")
            break
    for materia in materias:
        if materia["id"] == nota.get("materia_id"):
            self.materia.fijar(f"{materia['codigo']} - {materia['nombre']}")
            break
    self.valor.fijar(nota.get("valor", ""))
    self.tipo.fijar(nota.get("tipo", ""))
    self.periodo.fijar(nota.get("periodo", ""))

def guardar(self):
    etiqueta_estudiante = self.estudiante.obtener()
    etiqueta_materia = self.materia.obtener()
    if not etiqueta_estudiante:
        self.aviso.error("Selecciona un estudiante.")
        return
    if not etiqueta_materia:
        self.aviso.error("Selecciona una materia.")
        return

    datos = {
        "estudiante_id": self.etiquetas_estudiante[etiqueta_estudiante],
        "materia_id": self.etiquetas_materia[etiqueta_materia],
        "valor": self.valor.obtener(),
        "tipo": self.tipo.obtener(),
        "periodo": self.periodo.obtener(),
    }
    try:
        if self.editando:
            resultado = self.servicio.actualizar_nota(self.nota["id"], datos)
            mensaje = f"Nota de {resultado['valor']} actualizada."
        else:
            resultado = self.servicio.agregar_nota(datos)
            mensaje = (
                f"Nota registrada: {resultado['valor']} / 100 "
                f"({self.servicio.nombre_estudiante(resultado['estudiante_id'])}, "
                f"{self.servicio.codigo_materia(resultado['materia_id'])})"
            )
    except ErrorValidacion as error:
        self.aviso.error(str(error))
    return
    self.valores = {"id": resultado["id"], "mensaje": mensaje}
    self.destroy()

class DialogoContrasena(DialogoBase):
    def __init__(self, master, servicio):
        super().__init__(master, "Cambiar contrasena", ancho=480)
        self.servicio = servicio
        self.titulo(
            "Cambiar mi contrasena",
            "Debes confirmar la contrasena actual para poder entrar en el sistema.",
        )

marco = tk.Frame(self.cuerpo, bg="#ffffff")
marco.pack(fill="both", expand=True)

```

```
self.actual = widgets.CampoTexto(marco, "Contrasena actual", tipo="clave", a
self.nueva = widgets.CampoTexto(marco, "Nueva contrasena", tipo="clave", an
self.confirmacion = widgets.CampoTexto(marco, "Confirmar nueva contrasena",

for campo in (self.actual, self.nueva, self.confirmacion):
    campo.entrada.bind("<Return>", lambda _e: self.guardar())

self.actual.enfocar()
self.aceptar()

def guardar(self):
    try:
        self.servicio.cambiar_contrasena(
            self.actual.obtener(), self.nueva.obtener(), self.confirmacion.obtener()
        )
    except (ErrorValidacion, ErrorAutenticacion, ErrorPermiso) as error:
        self.aviso.error(str(error))
    return
    self.valores = {"mensaje": "Contrasena actualizada correctamente."}
    self.destroy()
```

## 10.11 Panel del docente

Ruta: [sistema\\_academico/interfaz/vista\\_docente.py](#)

*Tres pestanas (Estudiantes, Materias, Notas) con buscador, botones de agregar, editar y eliminar, y tarjetas con el resumen general.*

479 lineas

```

import tkinter as tk
from tkinter import messagebox, ttk

from interfaz import estilos, widgets
from interfaz.dialogos import DialogoContrasena, DialogoEstudiante, DialogoNota
from nucleo.servicios import ErrorPermiso, ErrorValidacion

NOTA_APROBACION = 60

class VistaDocente(ttk.Frame):
    def __init__(self, master, servicio, al_cerrar_sesion):
        super().__init__(master, style="TFrame")
        self.servicio = servicio
        self.al_cerrar_sesion = al_cerrar_sesion

        self.filtro_estudiantes_var = tk.StringVar()
        self.filtro_materias_var = tk.StringVar()
        self.filtro_notas_var = tk.StringVar()

        self._construir_cabecera()
        self._construir_cuerpo()

        self.filtro_estudiantes_var.trace_add("write", lambda *_: self._cargar_estudiantes())
        self.filtro_materias_var.trace_add("write", lambda *_: self._cargar_materias())
        self.filtro_notas_var.trace_add("write", lambda *_: self._cargar_notas())

        self.refrescar_todo()

    def _construir_cabecera(self):
        cabecera = ttk.Frame(self, style="Superior.TFrame")
        cabecera.pack(fill="x")
        interior = ttk.Frame(cabecera, style="Superior.TFrame")
        interior.pack(fill="x", padx=16, pady=10)

        ttk.Label(interior, text="Panel del docente", style="Superior.TLabel").pack(
            side="left",
            interior,
            text=f"Sesion: {self.servicio.sesion}",
            style="SuperiorRol.TLabel",
        ).pack(side="left", padx=14)

        acciones = ttk.Frame(interior, style="Superior.TFrame")
        acciones.pack(side="right")
        ttk.Button(acciones, text="Cambiar contrasena", command=self._cambiar_contrasena).pack(
            side="left",
        )
        ttk.Button(acciones, text="Cerrar sesion", command=self.al_cerrar_sesion).pack(
            side="left",
        )

    def _construir_cuerpo(self):
        self.tarjetas = ttk.Frame(self)
        self.tarjetas.pack(fill="x", padx=12, pady=(12, 0))
        self.tarjeta_estudiantes = widgets.TarjetaEstadistica(self.tarjetas, "Estudiantes")
        self.tarjeta_materias = widgets.TarjetaEstadistica(self.tarjetas, "Materias")
        self.tarjeta_notas = widgets.TarjetaEstadistica(self.tarjetas, "Notas registradas")
        self.tarjeta_promedio = widgets.TarjetaEstadistica(self.tarjetas, "Promedio")

        self.libro = ttk.Notebook(self)
        self.libro.pack(fill="both", expand=True, padx=12, pady=(10, 12))

        self.pestana_estudiantes = self._crear_pestana_estudiantes()
        self.pestana_materias = self._crear_pestana_materias()

```

```

self.pestana_notas = self._crear_pestana_notas()

def _crear_pestana_estudiantes(self):
    marco = ttk.Frame(self.libro, style="TFrame")
    self.libro.add(marco, text=" Estudiantes ")

    barra = ttk.Frame(marco)
    barra.pack(fill="x", padx=12, pady=(12, 4))
    ttk.Label(barra, text="Buscar:").pack(side="left")
    ttk.Entry(barra, textvariable=self.filtro_estudiantes_var, width=34, style="Bus
side="left", padx=6
    )
    ttk.Button(barra, text="Limpiar", command=lambda: self.filtro_estudiantes_va

    acciones = ttk.Frame(barra)
    acciones.pack(side="right")
    ttk.Button(acciones, text="+ Nuevo estudiante", command=self._nuevo_estudian
    ttk.Button(acciones, text="Editar", command=self._editar_estudiante).pack(s
    ttk.Button(acciones, text="Ver notas", command=self._ver_notas_estudiante).p
    ttk.Button(
    acciones, text="Restablecer clave", command=self._restablecer_clave
    ).pack(side="left", padx=4)
    ttk.Button(acciones, text="Eliminar", style="Peligro.TButton", command=self.
side="left", padx=4
    )

    self.tabla_estudiantes = widgets.Tabla(
    marco,
    columnas=("cedula", "nombres", "carrera", "email", "telefono", "usuario"),
    titulos=("Cedula", "Estudiante", "Carrera", "Email", "Telefono", "Usuario"),
    anchos=(110, 200, 200, 190, 110, 110),
    on_doble_clic=lambda _id: self._editar_estudiante(),
    )
    self.resumen_estudiantes = widgets.Aviso(marco)
    self.resumen_estudiantes.label.pack(anchor="w", padx=16, pady=(0, 8))
    return marco

def _crear_pestana_materias(self):
    marco = ttk.Frame(self.libro, style="TFrame")
    self.libro.add(marco, text=" Materias ")

    barra = ttk.Frame(marco)
    barra.pack(fill="x", padx=12, pady=(12, 4))
    ttk.Label(barra, text="Buscar:").pack(side="left")
    ttk.Entry(barra, textvariable=self.filtro_materias_var, width=34, style="Bus
side="left", padx=6
    )
    ttk.Button(barra, text="Limpiar", command=lambda: self.filtro_materias_var.s

    acciones = ttk.Frame(barra)
    acciones.pack(side="right")
    ttk.Button(acciones, text="+ Nueva materia", command=self._nueva_materia).pa
    ttk.Button(acciones, text="Editar", command=self._editar_materia).pack(side
    ttk.Button(acciones, text="Eliminar", style="Peligro.TButton", command=self.
side="left", padx=4
    )

    self.tabla_materias = widgets.Tabla(
    marco,
    columnas=("codigo", "nombre", "creditos", "semestre", "docente", "inscritos")

```

```

        titulos=("Codigo", "Materia", "Creditos", "Semestre", "Docente", "Notas"),
        anchos=(100, 300, 80, 80, 200, 70),
        on_doble_clic=lambda _id: self._editar_materia(),
    )
    return marco

    def _crear_pestana_notas(self):
        marco = ttk.Frame(self.libro, style="TFrame")
        self.libro.add(marco, text=" Notas ")

        barra = ttk.Frame(marco)
        barra.pack(fill="x", padx=12, pady=(12, 4))
        ttk.Label(barra, text="Buscar:").pack(side="left")
        ttk.Entry(barra, textvariable=self.filtro_notas_var, width=30, style="Busca
        side="left", padx=6
    )
    ttk.Button(barra, text="Limpiar", command=lambda: self.filtro_notas_var.set

    acciones = ttk.Frame(barra)
    acciones.pack(side="right")
    ttk.Button(acciones, text="+ Nueva nota", command=self._nueva_nota).pack(sic
    ttk.Button(acciones, text="Editar", command=self._editar_nota).pack(side="le
    ttk.Button(acciones, text="Eliminar", style="Peligro.TButton", command=self
    side="left", padx=4
    )

    self.tabla_notas = widgets.Tabla(
        marco,
        columnas=("estudiante", "materia", "valor", "tipo", "periodo", "fecha"),
        titulos=("Estudiante", "Materia", "Nota", "Tipo", "Periodo", "Fecha"),
        anchos=(220, 260, 70, 110, 90, 100),
        on_doble_clic=lambda _id: self._editar_nota(),
    )
    self.tabla_notas.arbol.tag_configure("aprobado", background="#eaf7ee")
    self.tabla_notas.arbol.tag_configure("reprobado", background="#fdecea")
    self.resumen_notas = widgets.Aviso(marco)
    self.resumen_notas.label.pack(anchor="w", padx=16, pady=(0, 8))
    return marco

    def refrescar_todo(self):
        if self.servicio.sesion is None:
            return
        try:
            estadisticas = self.servicio.estadisticas_generales()
        except Exception as error:
            messagebox.showerror(title="Error de datos", message=str(error), parent=self)
            return
        self.tarjeta_estudiantes.actualizar(estadisticas["estudiantes"])
        self.tarjeta_materias.actualizar(estadisticas["materias"])
        self.tarjeta_notas.actualizar(estadisticas["notas"])
        self.tarjeta_promedio.actualizar(f"{estadisticas['promedio_general']:.2f}")
        self._cargar_estudiantes()
        self._cargar_materias()
        self._cargar_notas()

    @staticmethod
    def _coincide(filtro, *campos):
        filtro = filtro.strip().lower()
        if not filtro:
            return True

```

```

return any(filtro in str(campo or "").lower() for campo in campos)

def _cargar_estudiantes(self):
    if not hasattr(self, "tabla_estudiantes"):
        return
    filtro = self.filtro_estudiantes_var.get() if hasattr(self, "filtro_estudiantes_var") else ""
    self.tabla_estudiantes.limpiar()
    visibles = 0
    estudiantes = []
    for e in self.servicio.estudiantes.listar():
        if all(clave in e for clave in ("id", "cedula", "nombres", "apellidos")):
            if not self._coincide(
                filtro,
                estudiante["cedula"],
                estudiante["nombres"],
                estudiante["apellidos"],
                estudiante.get("carrera", "-"),
                estudiante.get("email", "-"),
            ):
                continue
            usuario = next(
                (u for u in self.servicio.usuarios.listar() if u.get("estudiante_id") == estudiante["id"]),
                None,
            )
            self.tabla_estudiantes.insertar(
                estudiante["id"],
                (
                    estudiante["cedula"],
                    f"{estudiante['nombres']} {estudiante['apellidos']}",
                    estudiante.get("carrera", "-"),
                    estudiante.get("email", "-"),
                    estudiante.get("telefono", "-"),
                    usuario["nombre_usuario"] if usuario else "-",
                ),
            )
            visibles += 1
    total = len(estudiantes)
    self.resumen_estudiantes.info(f"Mostrando {visibles} de {total} estudiantes")

def _cargar_materias(self):
    if not hasattr(self, "tabla_materias"):
        return
    filtro = self.filtro_materias_var.get() if hasattr(self, "filtro_materias_var") else ""
    self.tabla_materias.limpiar()
    conteo = {}
    for nota in self.servicio.notas.listar():
        clave = nota.get("materia_id")
        if clave:
            conteo[clave] = conteo.get(clave, 0) + 1
    materias = [m for m in self.servicio.materias.listar() if "id" in m and "codigo" in m]
    for materia in sorted(materias, key=lambda m: m["codigo"]):
        if not self._coincide(
            filtro, materia.get("codigo", "-"), materia.get("nombre", "-"), materia.get("descripcion", "-")
        ):
            continue
        self.tabla_materias.insertar(
            materia["id"],

```

```

(
    materia.get("codigo", "-"),
    materia.get("nombre", "(sin nombre)"),
    materia.get("creditos", "-"),
    materia.get("semestre", "-"),
    materia.get("docente") or "-",
    conteo.get(materia["id"], 0),
),
)

def _cargar_notas(self):
    if not hasattr(self, "tabla_notas"):
        return
    filtro = self.filtro_notas_var.get() if hasattr(self, "filtro_notas_var") else ""
    self.tabla_notas.limpiar()
    if not hasattr(self, "filtro_notas_var"):
        return
    notas = []
    nota
    for nota in self.servicio.notas.listar():
        if all(clave in nota for clave in ("id", "estudiante_id", "materia_id", "valor")):
            notas.append(nota)
    notas = sorted(
        notas,
        key=lambda n: (n.get("periodo", ""), n.get("fecha", ""), n["id"]),
        reverse=True,
    )
    aprobadas = reprobadas = 0
    visibles = 0
    for nota in notas:
        estudiante = self.servicio.nombre_estudiante(nota["estudiante_id"])
        codigo = self.servicio.codigo_materia(nota["materia_id"])
        materia = self.servicio.nombre_materia(nota["materia_id"])
        if not self._coincide(filtro, estudiante, materia, codigo, nota["tipo"], nota["valor"]):
            continue
        aprobado = nota["valor"] >= NOTA_APROBACION
        aprobadas += aprobado
        reprobadas += not aprobado
        self.tabla_notas.insertar(
            nota["id"],
            (
                estudiante,
                f"{codigo} - {materia}",
                f"{nota['valor']:g}",
                nota["tipo"],
                nota["periodo"],
                nota.get("fecha", ""),
            ),
            etiqueta="aprobado" if aprobado else "reprobado",
        )
        visibles += 1
    if visibles:
        promedio = sum(n["valor"] for n in notas) / len(notas)
        self.resumen_notas.info(
            f"{visibles} notas visibles | promedio general: {promedio:.2f} | "
            f"aprobadas: {aprobadas} | reprobadas: {reprobadas}"
        )
    else:
        self.resumen_notas.info("No hay notas registradas todavia.")

```

```

def _seleccion(self, tabla, entidad):
    id_registro = tabla.id_seleccionada()
    if not id_registro:
        messagebox.showinfo(title="Selecciona un registro", message=f"Primero selecciona un registro")
    return id_registro

def _nuevo_estudiante(self):
    resultado = DialogoEstudiante(self, self.servicio).mostrar()
    if resultado:
        self.refrescar_todo()
        self.tabla_estudiantes.resaltado_si_existe(resultado["id"])
        self.resumen_estudiantes.exito(resultado["mensaje"].splitlines()[0])

def _editar_estudiante(self):
    id_registro = self._seleccion(self.tabla_estudiantes, "un estudiante")
    if not id_registro:
        return
    estudiante = self.servicio.estudiantes.buscar_por_id(id_registro)
    if estudiante is None:
        return
    resultado = DialogoEstudiante(self, self.servicio, estudiante).mostrar()
    if resultado:
        self.refrescar_todo()
        self.tabla_estudiantes.resaltado_si_existe(id_registro)
        self.resumen_estudiantes.exito(resultado["mensaje"])

def _eliminar_estudiante(self):
    id_registro = self._seleccion(self.tabla_estudiantes, "un estudiante")
    if not id_registro:
        return
    estudiante = self.servicio.estudiantes.buscar_por_id(id_registro)
    if estudiante is None:
        return
    notas = self.servicio.notas_de(id_registro)
    detalle = (
        f"\n\nTambien se eliminaran sus {len(notas)} notas registradas."
        if notas
        else "\n\nNo tiene notas registradas."
    )
    if not messagebox.askyesno(
        "Confirmar borrado",
        f"Eliminar a {estudiante['nombres']} {estudiante['apellidos']} y su usuario",
        icon="warning",
        parent=self,
    ):
        return
    try:
        self.servicio.eliminar_estudiante(id_registro)
    except ErrorPermiso as error:
        messagebox.showerror(title="Sin permisos", message=str(error), parent=self)
    return
    self.refrescar_todo()
    self.resumen_estudiantes.exito("Estudiante eliminado.")

def _restablecer_clave(self):
    id_registro = self._seleccion(self.tabla_estudiantes, "un estudiante")
    if not id_registro:
        return
    estudiante = self.servicio.estudiantes.buscar_por_id(id_registro)
    if estudiante is None:

```

```

return
if not messagebox.askyesno(
"Restablecer contrasena",
f"Generar una nueva contrasena para {estudiante['nombres']} {estudiante['apellidos']}",
parent=self,
):
return
try:
clave = self.servicio.restablecer_clave_estudiante(id_registro)
except (ErrorPermiso, ErrorValidacion) as error:
messagebox.showerror(title="No se pudo restablecer", message=str(error), parent=self)
return
messagebox.showinfo(
title="Nueva contrasena",
message=f"Usuario: {estudiante['nombres']} {estudiante['apellidos']}\nContrasena nueva: {clave}\n\nAtencion: La contrasena debe tener al menos 8 caracteres",
parent=self,
)

def _ver_notas_estudiante(self):
id_registro = self._seleccion(self.tabla_estudiantes, "un estudiante")
if not id_registro:
return
self.refrescar_todo()
self.filtro_notas_var.set(self.servicio.nombre_estudiante(id_registro))
self.libro.select(self.pestana_notas)
self._cargar_notas()

def _nueva_materia(self):
resultado = DialogoMateria(self, self.servicio).mostrar()
if resultado:
self.refrescar_todo()
self.tabla_materias.resaltado_si_existe(resultado["id"])

def _editar_materia(self):
id_registro = self._seleccion(self.tabla_materias, "una materia")
if not id_registro:
return
materia = self.servicio.materias.buscar_por_id(id_registro)
if materia is None:
return
if DialogoMateria(self, self.servicio, materia).mostrar():
self.refrescar_todo()
self.tabla_materias.resaltado_si_existe(id_registro)

def _eliminar_materia(self):
id_registro = self._seleccion(self.tabla_materias, "una materia")
if not id_registro:
return
materia = self.servicio.materias.buscar_por_id(id_registro)
if materia is None:
return
notas = [n for n in self.servicio.notas.listar() if n["materia_id"] == id_registro]
detalle = f"\n\nTambien se eliminaran sus {len(notas)} notas." if notas else ""
if not messagebox.askyesno(
"Confirmar borrado",
f"Eliminar la materia {materia['codigo']} - {materia['nombre']}?{detalle}",
icon="warning",
parent=self,
):
return

```

```

try:
    self.servicio.eliminar_materia(id_registro)
except ErrorPermiso as error:
    messagebox.showerror(title="Sin permisos", message=str(error), parent=self)
    return
self.refrescar_todo()

def _nueva_nota(self):
    if not self.servicio.estudiantes.listar() or not self.servicio.materias.listar():
        messagebox.showwarning(
            title="Faltan datos",
            message="Necesitas al menos un estudiante y una materia registrados para poder registrar una nota",
            parent=self,
        )
    return
    resultado = DialogoNota(self, self.servicio).mostrar()
    if resultado:
        self.refrescar_todo()
        self.tabla_notas.resaltado_si_existe(resultado["id"])
        self.resumen_notas.exito(resultado["mensaje"])

def _editar_nota(self):
    id_registro = self._seleccion(self.tabla_notas, "una nota")
    if not id_registro:
        return
    nota = self.servicio.notas.buscar_por_id(id_registro)
    if nota is None:
        return
    if DialogoNota(self, self.servicio, nota=nota).mostrar():
        self.refrescar_todo()
        self.tabla_notas.resaltado_si_existe(id_registro)

def _eliminar_nota(self):
    id_registro = self._seleccion(self.tabla_notas, "una nota")
    if not id_registro:
        return
    nota = self.servicio.notas.buscar_por_id(id_registro)
    if nota is None:
        return
    if not messagebox.askyesno(
        "Confirmar borrado",
        f"Eliminar la nota de {self.servicio.nombre_estudiante(nota['estudiante_id'])} en {self.servicio.codigo_materia(nota['materia_id'])} ({nota['valor']:g})?",
        icon="warning",
        parent=self,
    ):
        return
    try:
        self.servicio.eliminar_nota(id_registro)
    except ErrorPermiso as error:
        messagebox.showerror(title="Sin permisos", message=str(error), parent=self)
    return
    self.refrescar_todo()
    self.resumen_notas.exito("Nota eliminada.")

def _cambiar_contrasena(self):
    resultado = DialogoContrasena(self, self.servicio).mostrar()
    if resultado:
        messagebox.showinfo(title="Listo", message=resultado["mensaje"], parent=self)

```

## 10.12 Panel del estudiante

Ruta: [sistema\\_academico/interfaz/vista\\_estudiante.py](#)

*Solo lectura: sus notas, el promedio por materia y su ficha personal.*

199 líneas

```

import tkinter as tk
from tkinter import messagebox, ttk

from interfaz import estilos, widgets
from interfaz.dialogos import DialogoContrasena
from nucleo.servicios import ErrorPermiso

NOTA_APROBACION = 60

class VistaEstudiante(ttk.Frame):
    def __init__(self, master, servicio, al_cerrar_sesion):
        super().__init__(master, style="TFrame")
        self.servicio = servicio
        self.al_cerrar_sesion = al_cerrar_sesion
        self.id_estudiante = servicio.sesion.estudiante_id

        self._construir_cabecera()
        self._construir_cuerpo()
        self.refrescar()

    def _construir_cabecera(self):
        cabecera = ttk.Frame(self, style="Superior.TFrame")
        cabecera.pack(fill="x")
        interior = ttk.Frame(cabecera, style="Superior.TFrame")
        interior.pack(fill="x", padx=16, pady=10)

        ttk.Label(interior, text="Mis notas", style="Superior.TLabel").pack(side="left")
        ttk.Label(
            interior,
            text=f"Sesion: {self.servicio.sesion}",
            style="SuperiorRol.TLabel",
        ).pack(side="left", padx=14)

        acciones = ttk.Frame(interior, style="Superior.TFrame")
        acciones.pack(side="right")
        ttk.Button(acciones, text="Actualizar", command=self.refrescar).pack(side="left")
        ttk.Button(acciones, text="Cambiar contrasena", command=self._cambiar_contra).pack(side="left")
        ttk.Button(acciones, text="Cerrar sesion", command=self.al_cerrar_sesion).pack(side="left")

    def _construir_cuerpo(self):
        self.tarjetas = ttk.Frame(self)
        self.tarjetas.pack(fill="x", padx=12, pady=(12, 0))
        self.tarjeta_promedio = widgets.TarjetaEstadistica(self.tarjetas, "Promedio")
        self.tarjeta_aprobadas = widgets.TarjetaEstadistica(self.tarjetas, "Notas aprobadas")
        self.tarjeta_reprobadas = widgets.TarjetaEstadistica(self.tarjetas, "Notas reprobadas")
        self.tarjeta_total = widgets.TarjetaEstadistica(self.tarjetas, "Total de notas")

        self.libro = ttk.Notebook(self)
        self.libro.pack(fill="both", expand=True, padx=12, pady=(10, 12))

        self.pestana_detalle = ttk.Frame(self.libro, style="TFrame")
        self.pestana_materias = ttk.Frame(self.libro, style="TFrame")
        self.pestana_perfil = ttk.Frame(self.libro, style="TFrame")
        self.libro.add(self.pestana_detalle, text=" Todas mis notas ")
        self.libro.add(self.pestana_materias, text=" Por materia ")
        self.libro.add(self.pestana_perfil, text=" Mi perfil ")

        marco = ttk.Frame(self.pestana_detalle, style="TFrame")
        marco.pack(fill="both", expand=True)

```

```

self.tabla_notas = widgets.Tabla(
marco,
columnas=("materia", "valor", "tipo", "periodo", "fecha", "estado"),
titulos=("Materia", "Nota", "Tipo", "Periodo", "Fecha", "Estado"),
anchos=(280, 70, 110, 90, 100, 110),
)
self.tabla_notas.arbol.tag_configure("aprobado", background="#eaf7ee")
self.tabla_notas.arbol.tag_configure("reprobado", background="#fdecea")
self.resumen_notas = widgets.Aviso(marco)
self.resumen_notas.Label.pack(anchor="w", padx=16, pady=(0, 8))

marco_materias = ttk.Frame(self.pestana_materias, style="TFrame")
marco_materias.pack(fill="both", expand=True)
self.tabla_materias = widgets.Tabla(
marco_materias,
columnas=("materia", "promedio", "total", "creditos", "estado"),
titulos=("Materia", "Promedio", "Notas", "Creditos", "Estado"),
anchos=(300, 100, 80, 90, 120),
)
self.resumen_materias = widgets.Aviso(marco_materias)
self.resumen_materias.Label.pack(anchor="w", padx=16, pady=(0, 8))

self._construir_perfil()

def _construir_perfil(self):
marco = ttk.Frame(self.pestana_perfil, style="TFrame")
marco.pack(fill="both", expand=True)
tarjeta = ttk.LabelFrame(marco, text="Datos personales", style="Tarjeta.TLabel")
tarjeta.pack(fill="x", padx=16, pady=16)

estudiante = self.servicio.estudiantes.buscar_por_id(self.id_estudiante)
usuario = next(
(u for u in self.servicio.usuarios.listar() if u.get("estudiante_id") == self.id_estudiante)
)

campos = []
if estudiante:
campos = [
("Cedula / Matricula", estudiante["cedula"]),
("Nombres", estudiante["nombres"]),
("Apellidos", estudiante["apellidos"]),
("Carrera", estudiante["carrera"]),
("Email", estudiante["email"]),
("Telefono", estudiante["telefono"]),
("Fecha de registro", estudiante.get("fecha_registro", "-")),
("Usuario de acceso", usuario["nombre_usuario"] if usuario else "-"),
]
else:
campos = [("Aviso", "No se encontro tu ficha de estudiante.")]

for indice, (etiqueta, valor) in enumerate(campos):
fila = tk.Frame(tarjeta, bg="ffffff")
fila.pack(fill="x", pady=4)
ttk.Label(fila, text=etiqueta, style="Campo.TLabel", width=22).pack(side="left")
ttk.Label(fila, text=str(valor), style="Campo.TLabel", foreground="#1f2a33").pack(side="left")
if indice == 0:
marco.columnconfigure(0, weight=1)

nota = tk.Label(

```

```

marco,
text="Aquí solo puedes consultar tus calificaciones. Para corregirlas, comun
bg=estilos.GRIS_FONDO,
fg="#8a8a8a",
font=("Segoe UI", 9),
)
nota.pack(anchor="w", padx=20, pady=(0, 10))

def refrescar(self):
if self.servicio.sesion is None:
return
if not self.servicio.estudiantes.buscar_por_id(self.id_estudiante):
messagebox.showwarning(
title="Ficha no encontrada",
message="Tu usuario no tiene una ficha de estudiante asociada. Contacta a la
parent=self,
)

resumen = self.servicio.resumen_estudiante(self.id_estudiante)
self.tarjeta_promedio.actualizar(f"{resumen['promedio']:.2f}")
self.tarjeta_aprobadas.actualizar(resumen["aprobadas"])
self.tarjeta_reprobadas.actualizar(resumen["reprobadas"])
self.tarjeta_total.actualizar(resumen["total"])

notas = self.servicio.notas_de(self.id_estudiante)
self.tabla_notas.limpiar()
for nota in notas:
aprobado = nota["valor"] >= NOTA_APROBACION
self.tabla_notas.insertar(
nota["id"],
(
f"{self.servicio.codigo_materia(nota['materia_id'])} - "
f"{self.servicio.nombre_materia(nota['materia_id'])}",
f"{nota['valor']:g}",
nota["tipo"],
nota["periodo"],
nota.get("fecha", ""),
"Aprobado" if aprobado else "Reprobado",
),
etiqueta="aprobado" if aprobado else "reprobado",
)

if notas:
promedios = self.servicio.promedios_por_materia(self.id_estudiante)
self.resumen_notas.info(
f"{len(notas)} notas registradas | promedio: {resumen['promedio']:.2f} | "
f"aprobadas: {resumen['aprobadas']} | reprobadas: {resumen['reprobadas']}"
)
else:
self.resumen_notas.info("Todavía no tienes notas registradas.")

self.tabla_materias.limpiar()
promedios = self.servicio.promedios_por_materia(self.id_estudiante)
for id_materia, info in sorted(
promedios.items(), key=lambda par: self.servicio.codigo_materia(par[0])
):
materia = self.servicio.materia_por_id(id_materia) or {}
aprobado = info["promedio"] >= NOTA_APROBACION
self.tabla_materias.insertar(
id_materia,

```

```
(
    f"{self.servicio.codigo_materia(id_materia)} - {self.servicio.nombre_materia(id_materia)}",
    f"{info['promedio']:.2f}",
    info["total"],
    materia.get("creditos", "-"),
    "Aprobado" if aprobado else "Reprobado",
),
)
if promedios:
    self.resumen_materias.info(
        f"{len(promedios)} materias con notas registradas. El minimo para aprobar es {min(promedios)}"
    )
else:
    self.resumen_materias.info("Aun no hay notas por materia.")

def _cambiar_contrasena(self):
    resultado = DialogoContrasena(self, self.servicio).mostrar()
    if resultado:
        messagebox.showinfo(title="Listo", message=resultado["mensaje"], parent=self)
```

## 10.13 Punto de entrada

Ruta: [sistema\\_academico/principal.py](#)

*Crea la ventana principal, aplica el tema, decide que vista mostrar y maneja los errores de arranque.*

92 lineas

```

import sys
import tkinter as tk
from tkinter import messagebox

from config import TITULO_APP, VERSION
from interfaz import estilos
from interfaz.vista_docente import VistaDocente
from interfaz.vista_estudiante import VistaEstudiante
from interfaz.vista_login import VistaLogin
from nucleo.almacen import ArchivoInvalido
from nucleo.servicios import Servicio

class Aplicacion:
    def __init__(self):
        self.servicio = Servicio()
        self.raiz = tk.Tk()
        self.raiz.title(f"{TITULO_APP} v{VERSION}")
        self.raiz.geometry("1180x720")
        self.raiz.minsize(1000, 620)
        self.raiz.configure(bg=estilos.GRIS_FONDO)

        estilos.aplicar_estilos(self.raiz)
        self._icono()

        self.contenedor = tk.Frame(self.raiz, bg=estilos.GRIS_FONDO)
        self.contenedor.pack(fill="both", expand=True)

        self.raiz.protocol("WM_DELETE_WINDOW", self.cerrar)
        self._mostrar_login()

    def _icono(self):
        try:
            self.raiz.iconname(TITULO_APP)
        except tk.TclError:
            pass

    def _limpiar(self):
        for hijo in self.contenedor.winfo_children():
            hijo.destroy()

    def _mostrar_login(self, mensaje=None):
        self._limpiar()
        vista = VistaLogin(self.contenedor, self.servicio, self._al_entrar)
        vista.pack(fill="both", expand=True)
        if mensaje:
            messagebox.showinfo(title="Sesion cerrada", message=mensaje, parent=self.raiz)
        self.raiz.deiconify()

    def _al_entrar(self, sesion):
        self._limpiar()
        if sesion.es_docente:
            VistaDocente(self.contenedor, self.servicio, self._cerrar_sesion).pack(fill="both", expand=True)
        else:
            VistaEstudiante(self.contenedor, self.servicio, self._cerrar_sesion).pack(fill="both", expand=True)
        )

    def _cerrar_sesion(self):
        if messagebox.askyesno(

```

```

"Cerrar sesion", "Quieres salir de la cuenta actual?", icon="question", parent=self.root
):
self.servicio.cerrar_sesion()
self._mostrar_login("Sesion cerrada correctamente.")

def cerrar(self):
self.servicio.cerrar_sesion()
self.raiz.destroy()

def ejecutar(self):
self.raiz.mainloop()

def principal():
try:
Aplicacion().ejecutar()
except ArchivoInvalido as error:
root = tk.Tk()
root.withdraw()
messagebox.showerror(title="Error de almacenamiento", message=str(error), parent=root)
root.destroy()
return 1
except Exception as error:
root = tk.Tk()
root.withdraw()
messagebox.showerror(title="Error inesperado", message=f"{type(error).__name__}: {error}")
root.destroy()
return 1
return 0

if __name__ == "__main__":
sys.exit(principal())

```

# FIN DEL PROYECTO

Recorriste el Sistema Academico completo: la estructura, la guia paso a paso, los permisos por rol y el codigo de los 13 archivos del proyecto. Para ponerlo en marcha sigue el punto 3 de la portada.

*Python + Tkinter + archivos JSON. Sin base de datos y sin instalar librerias para que la aplicacion funcione.*